Collectors, groupingBy and Reduction

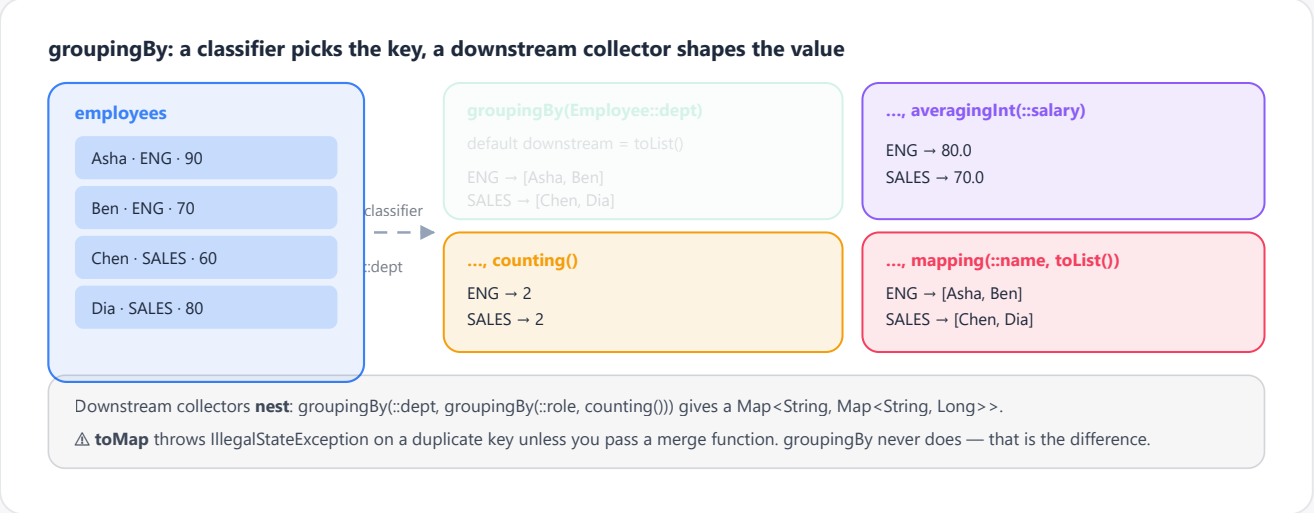
8 min read Updated 28 Jul 2026 Topic 33 of 65

Why it matters

`collect` is where streams stop being a nicer loop and start replacing whole methods. "Group these orders by customer and sum the totals" is fifteen lines of map-fiddling with `HashMap` and `computeIfAbsent`, or one line with `groupingBy`. Interviewers ask because knowing the collector vocabulary is the difference between using streams and being fluent in them.

The concept, simply

A `Collector` is a recipe for accumulating stream elements into a result. `Collectors` is the factory class of ready-made ones, and they **compose** — most take a *downstream* collector that shapes the values.



The classifier picks the key; the downstream collector decides what the value looks like. Swap the downstream and the same grouping produces lists, counts, averages or projections.

How it works

The collectors worth knowing by heart:

Collector	Produces
`toList()` / `toSet()`	A `List` / `Set`
`toMap(k, v)`	A `Map` — **throws on duplicate keys** without a merge function
`groupingBy(classifier)`	`Map<K, List<T>>`
`groupingBy(classifier, downstream)`	`Map<K, ?>` — the downstream shapes the value
`partitioningBy(predicate)`	`Map<Boolean, List<T>>` — always both keys
`counting()`, `summingInt`, `averagingInt`	Aggregates, as downstreams
`mapping(fn, downstream)`	Transforms before collecting
`joining(sep, prefix, suffix)`	A `String`
`teeing(c1, c2, merger)`	Two collectors in one pass (Java 12+)

`reduce` **versus** `collect` is the conceptual split. `reduce` is for *immutable* folding — combining values into a single new value, like summing. `collect` is for *mutable* accumulation — filling a container. Using `reduce` with a mutable accumulator technically works and is wrong: it copies on every element, making it quadratic, and it breaks under parallelism.

Code examples

The grouping vocabulary, which covers most real reporting needs:

JAVA

Copy

```java
// Group into lists
Map<String, List<Employee>> byDept =
    staff.stream().collect(groupingBy(Employee::dept));

// Count per group
Map<String, Long> countByDept =
    staff.stream().collect(groupingBy(Employee::dept, counting()));

// Sum and average per group
Map<String, Integer> payrollByDept =
    staff.stream().collect(groupingBy(Employee::dept, summingInt(Employee:

// Project before collecting — names only, not whole objects
Map<String, List<String>> namesByDept =
    staff.stream().collect(groupingBy(Employee::dept,
                           mapping(Employee::name, toList())));

// Highest earner per department
Map<String, Optional<Employee>> topByDept =
    staff.stream().collect(groupingBy(Employee::dept,
                           maxBy(comparingInt(Employee::salary))));

// Nested grouping — two levels, one pass
Map<String, Map<Boolean, List<Employee>>> byDeptThenSenior =
    staff.stream().collect(groupingBy(Employee::dept,
                           partitioningBy(e -> e.salary() > 80)));

// Choose the map implementation when order matters
Map<String, List<Employee>> sorted =
    staff.stream().collect(groupingBy(Employee::dept, TreeMap::new, toList
```

The `toMap` trap, which catches almost everyone once:

JAVA

Copy

```java
// ✗ IllegalStateException: Duplicate key ENG
Map<String, String> byDept =
    staff.stream().collect(toMap(Employee::dept, Employee::name));
```

```
// ... a merge function saying what to do with collisions.
Map<String, String> merged =
    staff.stream().collect(toMap(Employee::dept, Employee::name,
                                 (a, b) -> a + ", " + b));

// ⚠ A second trap: toMap uses HashMap.merge internally, which REMOVES the
//   entry if the merge function returns null — and it throws
//   NullPointerException if a VALUE is null. groupingBy has neither probl
//   When values may be null, prefer groupingBy or handle it explicitly.
```

`reduce` used properly — and improperly:

JAVA Copy

```
// ✓ Immutable folding, which is what reduce is for.
int total = staff.stream().mapToInt(Employee::salary).sum();        // best
Optional<Employee> top = staff.stream().max(comparingInt(Employee::salary)
int product = nums.stream().reduce(1, (a, b) -> a * b);

// ✗ Mutable accumulation via reduce — quadratic and parallel-unsafe.
List<String> names = staff.stream()
    .reduce(new ArrayList<String>(),
            (list, e) -> { list.add(e.name()); return list; },      // muta
            (a, b) -> { a.addAll(b); return a; });

// ✓ Use collect for mutable containers — that is the distinction.
List<String> names = staff.stream().map(Employee::name).toList();
```

Statistics and the Java 12+ addition worth knowing:

JAVA Copy

```
// One pass, five numbers.
IntSummaryStatistics stats = staff.stream().mapToInt(Employee::salary)
                                  .summaryStatistics();
stats.getCount(); stats.getSum(); stats.getMin(); stats.getAverage(); stat

// teeing — two collectors over one traversal (Java 12+).
record Report(long count, int total) { }
```

```
// Java 16+: toList() directly on the stream — shorter, and it returns an
// UNMODIFIABLE list, unlike collect(Collectors.toList()).
List<String> names = staff.stream().map(Employee::name).toList();
```

Under the hood

A `Collector` is four functions plus a set of characteristics: a **supplier** that creates a container, an **accumulator** that folds one element in, a **combiner** that merges two containers (used only in parallel), and a **finisher** that converts the container to the final result. Writing one by hand is rare, but naming the four parts is a common follow-up question.

`Collectors.toList()` **versus** `Stream.toList()` is a small distinction with real consequences. The collector returns a mutable `ArrayList` and permits nulls; `Stream.toList()` (Java 16+) returns an **unmodifiable** list — though it does allow null elements, unlike `List.of`. Code that collected and then mutated will break when someone "modernises" it to `toList()`, which is worth knowing before you do the refactor.

`partitioningBy` **versus** `groupingBy(x -> boolean)`**:** partitioning always returns both `true` and `false` keys even when one side is empty, and it uses a specialised two-slot map. Grouping on a boolean gives you a `HashMap` that may be missing a key entirely — a null-check waiting to happen.

Grouping is single-pass. A common instinct is to filter three times to produce three groups; `groupingBy` does it in one traversal. On a large or expensive source that is the whole game.

Common mistakes

- `toMap` **without a merge function.** Throws on the first duplicate key.
- `toMap` **with null values.** Throws `NullPointerException`. Use `groupingBy` or filter first.

- **Mutating an external collection in `forEach`.** Use a collector; it is correct in parallel too.
- **Assuming `Stream.toList()` is mutable.** It is not, unlike `Collectors.toList()`.
- **Filtering repeatedly instead of grouping.** One `groupingBy` beats three passes.
- **Relying on the map type from `groupingBy`.** It is a `HashMap` — pass `TreeMap::new` if you need order.
- **Boxing before aggregating.** `mapToInt( ... ).sum()` beats `collect(summingInt( ... ))` for a plain total.

Interview lens

Q: "Group employees by department and count them."
A: `staff.stream().collect(groupingBy(Employee::dept, counting()))`, giving a `Map<String, Long>`. The general shape is that the first argument classifies into keys and the optional second — the downstream collector — decides what each value looks like: `toList()` by default, or `counting()`, `summingInt`, `mapping`, even another `groupingBy`.

Q: "What is the difference between `reduce` and `collect`?"
A: `reduce` is immutable folding — combining values into a single new value, like a sum or a max. `collect` is mutable accumulation into a container. Using `reduce` with a mutable accumulator appears to work but copies on every element, which is quadratic, and it is unsafe in parallel because multiple threads would mutate the same container.

Q: "Why does `toMap` throw `IllegalStateException`?"
A: Duplicate keys. The two-argument form has no way to decide which value wins, so it refuses rather than silently dropping data. Pass a third merge-function argument to say what should happen — keep the first, keep the last, or combine

Q: "`partitioningBy` or `groupingBy` on a boolean?"
A: `partitioningBy`, because it guarantees both `true` and `false` keys exist even when one side is empty, and it uses a specialised two-entry map rather than a `HashMap`. Grouping on a boolean can return a map missing one key, which turns into a null check at every call site.

Q: "What are the parts of a `Collector`?"
A: A supplier that creates the container, an accumulator that adds one element, a combiner that merges two containers — only used in parallel — and a finisher that produces the final result, plus characteristics like `UNORDERED` or `IDENTITY_FINISH`. You rarely implement one directly, since `Collectors.of` and the built-ins cover almost everything.

Q: "`Collectors.toList()` or `Stream.toList()`?"
A: `Stream.toList()` from Java 16 is shorter and returns an unmodifiable list, which is usually what you want. The difference matters when existing code collected and then mutated the result — that breaks on the newer method. It also permits null elements, unlike `List.of`, which occasionally surprises people going the other way.

Cheat-card

- `groupingBy(classifier[, downstream])` — downstream shapes the value; **they nest**.
- Downstreams: `counting()` · `summingInt` · `averagingInt` · `mapping` · `maxBy` · `joining`.
- `toMap` **throws on duplicate keys** — pass a merge function. Also NPEs on null values.

- `partitioningBy` always yields **both** boolean keys.
- `reduce` = immutable fold · `collect` = mutable accumulation. **Never reduce into a container.**
- Collector = supplier + accumulator + combiner + finisher.
- `Stream.toList()` (16+) is **unmodifiable**; `Collectors.toList()` is not.
- Need order? `groupingBy(fn, TreeMap::new, toList())`.
- One pass for many stats: `summaryStatistics()` or `teeing` (12+).

QUICK CHECK — no marks, just for you.

1. Why does Collectors.toMap throw IllegalStateException?

- ◯ Because the stream was empty
- ◯ Duplicate keys — the two-argument form has no way to decide which value wins, so it refuses rather than silently dropping data
- ◯ Because the values were not Comparable
- ◯ Because the map exceeded its capacity

Check answer

2. What is the difference between reduce and collect?

- ◯ reduce is parallel and collect is sequential
- ◯ They are equivalent; collect is just newer
- ◯ reduce is for immutable folding into a single value; collect is for mutable accumulation into a container — using reduce with a mutable accumulator copies per element and breaks under parallelism
- ◯ collect can only produce Lists

3. Why prefer partitioningBy(predicate) over groupingBy on a boolean?

- ◯ It is the only one that preserves encounter order
- ◯ It allows more than two groups
- ◯ partitioningBy is the only one that compiles with a predicate
- ◯ It guarantees both true and false keys exist even when one side is empty, and uses a specialised two-entry map rather than a HashMap

Check answer

‹ Previous
Streams: Laziness and the Pipeline

Next ›
Parallel Streams: When They Help and When They Hurt

LEARN MORE

ARTICLES & DOCS

- Collectors — API docs
  docs.oracle.com
- Collector — API docs
  docs.oracle.com

VIDEOS



Streams API Tutorial in Java 8+
Daily Code Buffer

Hand-picked references for this lesson.



Java Interview Kit